Blasters By Airzone

NarfduinoBridge

This library implements the software controls required to operate a Half-H-Bridge on a Narfduino board. This can also be used to control other half-h-bridges wired in a similar way.

Software features include:

- Dead-time generation
- Shoot-through lock-out
- Simple Start and Stop functions
- PWM speed control
- Asynchronous processing
- Jam detection with emergency stop supported. Can be disabled.

Constructors

NarfduinoBridge() Default constructor. Will define the Run pin as D5 and the Stop pin as D15

NarfduinoBridge(byte RunPin, byte StopPin) Alternate constructor. You can define your own pins. In order to support speed control, RunPin should be a PWM enabled pin.

Setup, Configuration, Status

bool Init() Initialisation function. Run once only. Will return false in the rare case of failure.

void EnableAntiJam() Enables the Anti-Jam feature. In this state, if the bridge is running, and a PusherHeartbeat is not received within a defined timeframe, a jam will be considered to have occurred. The bridge will stop, and will need to be reset before it will operate again
Default: Enabled

void DisableAntiJam() Disables the Anti-Jam feature. In this state, a running bridge will run until it's told to stop. This is better suited to a flywheel cage with speed control and braking. In the event of a jam, if your software does not handle this, you will likely burn out your motor, and possibly the Narfduino.

bool HasJammed() Returns true if a jam was detected and the bridge is locked out. Returns false otherwise.

void SetBridgeSpeed(byte NewBridgeSpeed) Sets the PWM bridge speed. NewBridgeSpeed is a byte with a value between 0 and 100. 100 means full power. 0 means no power.
Default: 0

byte GetBridgeSpeed() Returns the current Bridge speed.

Operation

void StartBridge() Signals the bridge to start running. You should only run this when you want to change running state.

void StopBridge() Signals the b ridge to stop running. You should only run this when you want to change running state.

void ResetJam() Reset a Jam lockout condition. After the reset occurs, you will need to start the bridge again.

void PusherHeartbeat() Use to indicate that the pusher switch has been hit. This is the cue to the Anti-Jam feature that a jam has not occurred yet. Only required when using the Anti-Jam feature.

void ProcessBridge() Performs all bridge status processing. You should run this frequent and often. This will perform the state changes, dead-time generation, and jam detection. Ideally, run this every time in your main loop.

Override Definitions

#define _NARFDUINO_BRIDGE_ON_TRANSITION_TIME 10
10 millisecond dead-time from Running to Idle.

#define _NARFDUINO_BRIDGE_OFF_TRANSITION_TIME 2
2 millisecond dead-time from Idle to Running.

#define _NARFDUINO_PUSHER_MAX_CYCLE_TIME 500
500 millisecond maximum time between heartbeats for Jam detection. Note that this rate requires at least 2 – 3 DPS in order to be reliable.

Example

This example shows the use of NarfduinoBridge in operating a set of flywheels. Anti-Jam will be disabled. Every 3 seconds, the bridge will toggle running state, and on the next bridge start, the speed of the bridge will increase until it resets back to a slow speed.

```
// Example of NarfduinoBridge for flywheel / non-return pusher use
// The demo will spin the flywheels at 33% for 3 seconds, then stop for 3 second, then
66% for 3 seconds, then stop for 3 seconds, then 100% for 3 seconds, then stop for 3
seconds, and repeat.

// Include the library
#include "NarfduinoBridge.h"

// Create our bridge object
NarfduinoBridge Flywheels = NarfduinoBridge();
// n.b. You can also use NarfduinoBriodge( x, y ); where x is the PWM enabled pin for
"Go" (low side fet) and y is the digital pin for "Stop" (high side fet)


void setup() {
  Serial.begin( 57600 );

  // Initialise the library
  Flywheels.Init();

  // Since we are running in flywheel mode, you need to disable the jam detection.
  Flywheels.DisableAntiJam();
}

void loop() {
  // Load in some variables here.
  static byte DesiredSpeed = 100;
  static bool IsRunning = false;
  static unsigned long LastChange = 0;

  // Change state every 3 seconds
  if( millis() - LastChange > 3000 )
  {
    Serial.println( "Changing State Now");
    LastChange = millis();


    // Every 3 seconds, change the state of the bridge.
    // Every time it runs, run it 33% faster than before, and start again when you cap
100%
    if( IsRunning )
    {
      Serial.println( "Stop Bridge");
      // Signal the bridge to stop.
      Flywheels.StopBridge();

      // Update our tracking flags
      IsRunning = false;
    }
    else // Flywheels are stopped, start them now.
    {
      // Update our tracking flags
      IsRunning = true;

      // Prepare a new speed for the next time the
      if( DesiredSpeed == 15 )
        DesiredSpeed = 30;
      else if( DesiredSpeed == 30 )
        DesiredSpeed = 45;
      else
        DesiredSpeed = 15;

      Serial.print( "New Bridge Speed = " );
      Serial.println( DesiredSpeed );
      // Configure a new speed
      Flywheels.SetBridgeSpeed( DesiredSpeed );

      Serial.println( "Start Bridge");
      // Signal the bridge to run
      Flywheels.StartBridge();
    }
  }

  // Run this basically every loop. It will control the dead time
  Flywheels.ProcessBridge();

}
```

To search type and hit enter

RECENT POSTS

Caronavirus Update mk2
Caronavirus Update
The Mysterious Art of ESC's
Retrofitting Narfduino into other Software Defined Blasters
Using Narfduino to drive Flywheels

CATEGORIES

Narfduino
Nerf Hardware
Programming
Uncategorized

ARCHIVES

July 2020
March 2020
December 2019
July 2019

META

Log in
Entries feed
Comments feed
WordPress.org